

CHAPTER 15

IMAGE SEGMENTATION

WHAT WILL WE LEARN?

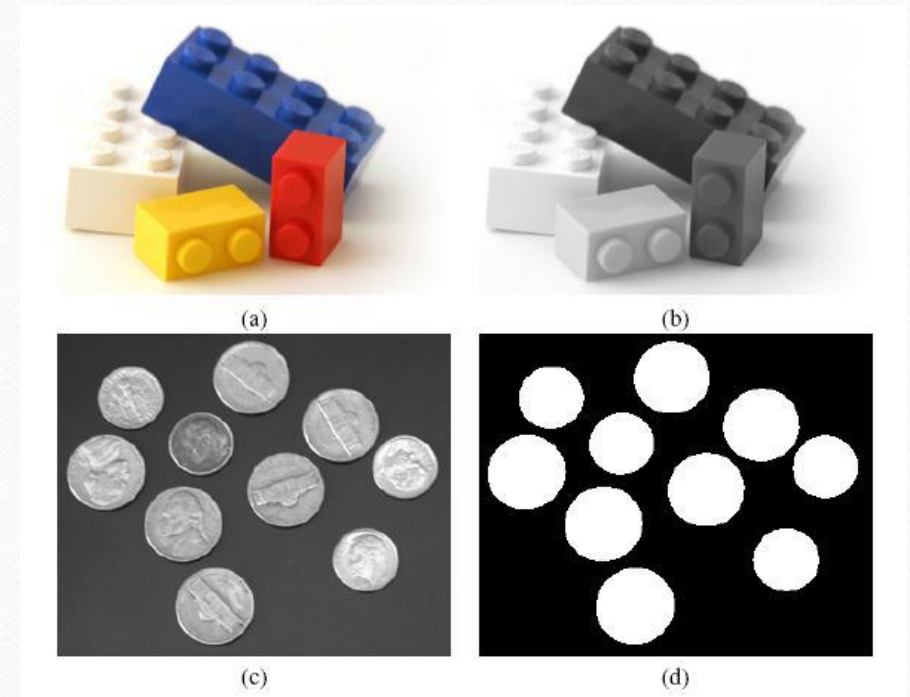
- What is image segmentation and why is it relevant?
- What is image thresholding and how is it implemented in MATLAB?
- What are the most commonly used image segmentation techniques and how do they work?

Introduction

- **Segmentation: Partitioning** an image into a set of **non-overlapping regions** whose union is the entire image.
 - These regions should ideally correspond to objects and their meaningful parts, and background.
- Based on one of two basic properties that can be extracted from pixel values -- **discontinuity** and **similarity** -- or a combination of them.
- One of the most crucial tasks in image processing and computer vision.
- **Still an open research problem.**

Introduction

- Segmentation: easy and hard images



Introduction

- There is no underlying theory of image segmentation, only ad hoc methods, whose performance is often evaluated indirectly, based on the performance of the larger system to which they belong.
- Image segmentation techniques can vary widely according to:
 - type of image (e.g., binary, gray, color)
 - choice of mathematical framework (e.g., morphology, image statistics, graph theory)
 - type of features (e.g., intensity, color, texture, motion) and
 - approach (e.g., top-down, bottom-up, graph-based).

Introduction

- There is no universally accepted taxonomy for classification of image segmentation algorithms, either.
- In the book, we use three categories:
 - **Intensity-based** (non-contextual) methods: work based on pixel distributions (i.e., histograms).
Example: thresholding.
 - **Region-based** (contextual) methods: rely on adjacency and connectivity criteria between a pixel and its neighbors.
Examples: region growing and split-and-merge.
 - **Other methods**: these include segmentation based on texture, edges, and motion, among others.

Intensity-based segmentation

- Rely on pixel statistics (histogram properties) to determine which pixels belong to foreground objects and which pixels should be labeled as background.
 - The simplest method within this category is *image thresholding*.

Image thresholding

- **Def.:** The conversion of an image with many gray levels into another image with fewer gray levels, usually only two.
- This conversion is usually performed by **comparing** each pixel intensity against a **reference value** (threshold) and **replacing the pixel with a value that means 'white' or 'black'** depending on the outcome of the comparison.

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{otherwise} \end{cases}$$

Image thresholding

- Global thresholding: single value of T for the whole image
- Steps:
 1. Inspect the image's histogram (using **imhist**)
 2. Select an appropriate value for T
 3. Apply the selected value (using **im2bw**) to the image
 4. Inspect the results: if they are acceptable, save resulting image; otherwise, make adjustments and repeat steps 2-4

Image thresholding

- Global thresholding: a procedure for selecting the best value of T automatically (Gonzalez & Woods, 2008)

```
Id = im2double(I); % I is a uint8 grayscale image
T = 0.5*(min(Id(:)) + max(Id(:)));
deltaT = 0.01; % convergence criterion
done = false;
while ~done
    g = Id >= T;
    Tnext = 0.5*(mean(Id(g)) + mean(Id(~g)));
    done = abs(T - Tnext) < deltaT;
    T = Tnext;
end
```


Image thresholding

- Global thresholding example

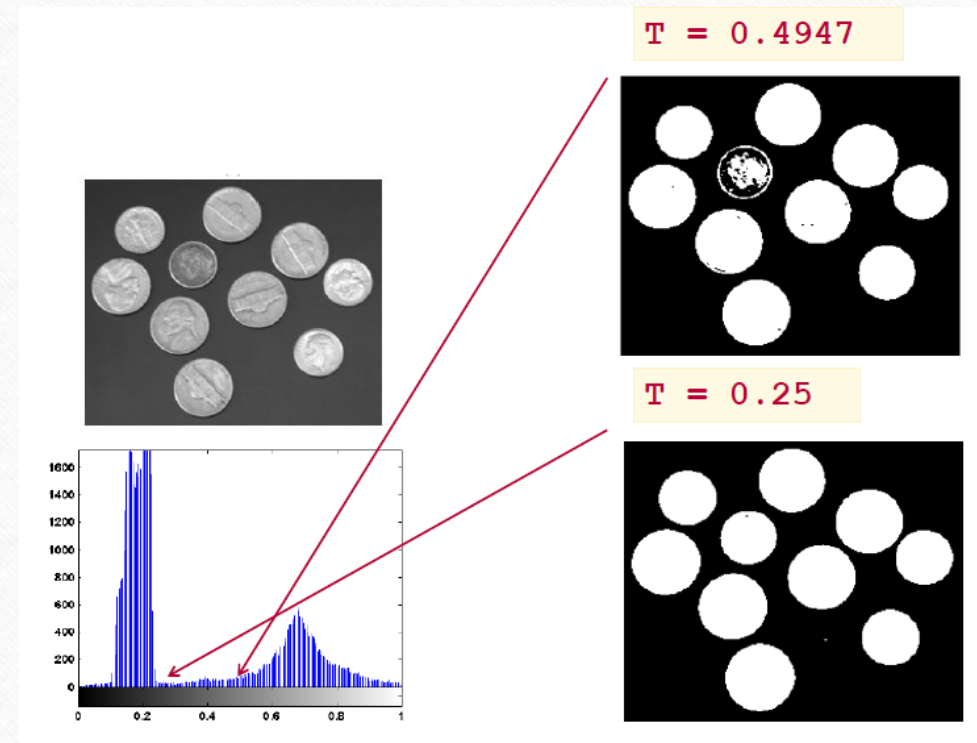


Image thresholding

- Optimal thresholding
 - (Otsu, 1979), implemented in the IPT as: **graythresh**.
 - For the previous example, provides the optimal value $T = 0.4941$

Image thresholding

- The role of illumination and noise
 - Even easy images may become hard to segment...

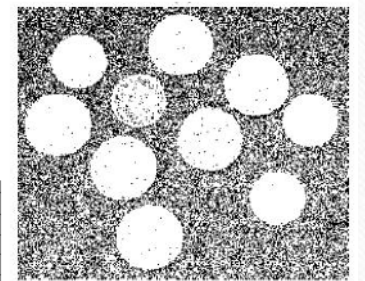
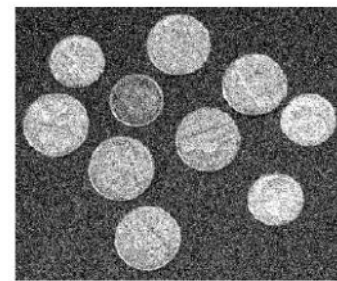
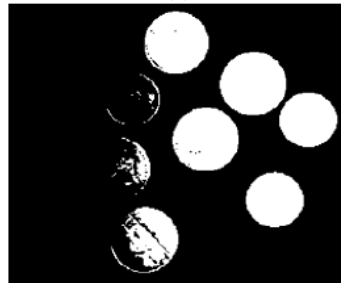
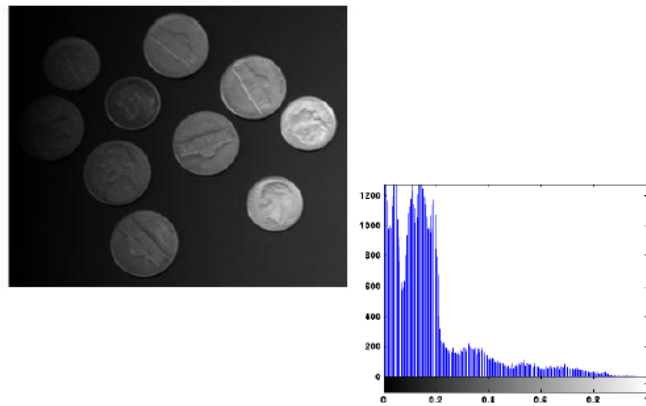
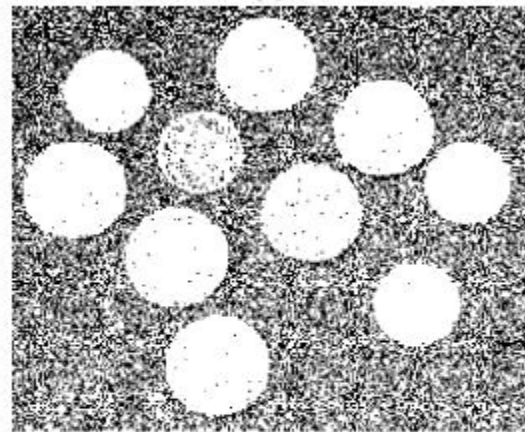
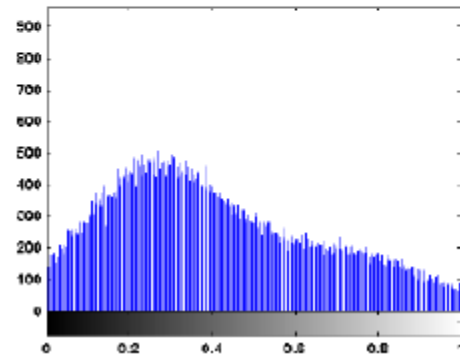
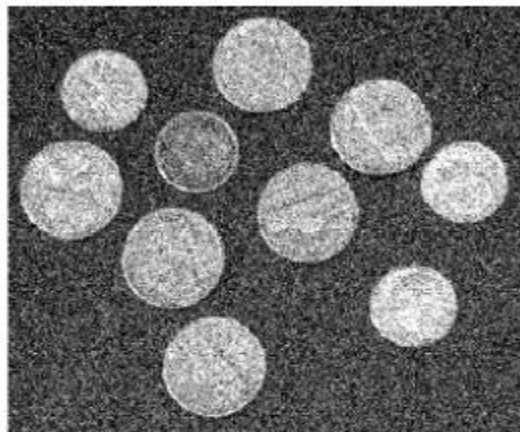


Image thresholding

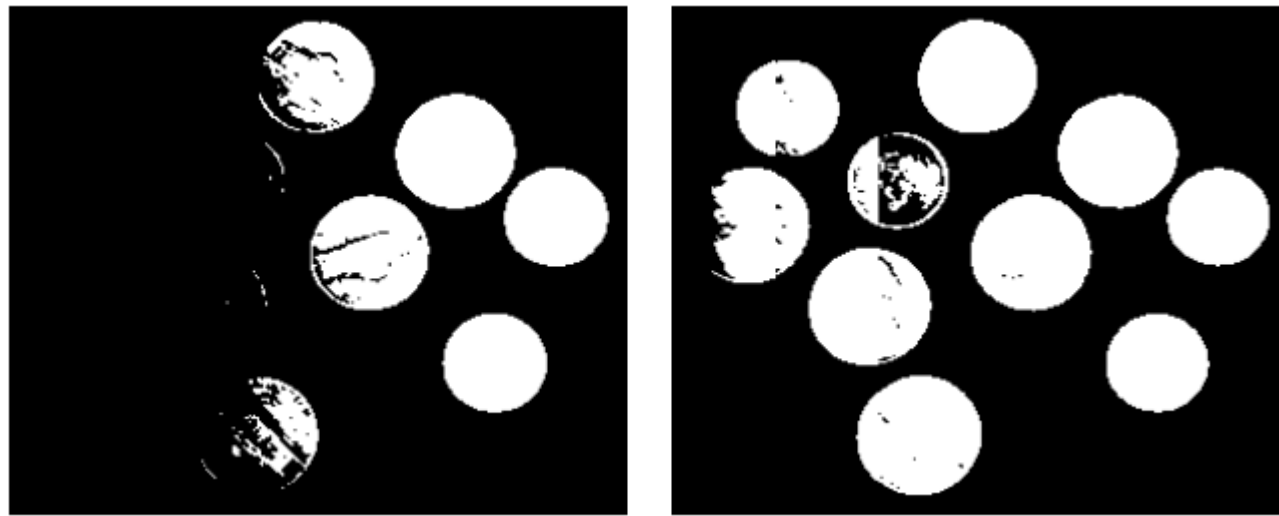


Local thresholding

- Local (adaptive) thresholding uses block processing to threshold blocks of pixels, one block at a time.
 - The size of the block is usually specified by the user.
 - Trade-off:
 - If the blocks that are too small: large computational cost.
 - If the blocks that are too large: results may not be substantially better than the ones obtained with global thresholding.

Local thresholding

- Example



Region-based segmentation

- Based on the fact that a pixel cannot be considered a part of an object or not based solely on its gray value.
- Incorporate measures of connectivity among pixels in order to decide whether those pixels belong to the same region (or object) or not.

Region-based segmentation

- Divide an image I into n regions $R_1, R_2, \dots, R_n$:

1. $\bigcup_{i=1}^n R_i = I$

2. R_i is a connected region, $i = 1, 2, \dots, n$.

3. $R_i \cap R_j = \emptyset$ for all i and $j, i \neq j$.

4. $P(R_i) = \text{TRUE}$ for $i = 1, 2, \dots, n$.

5. $P(R_i \cup R_j) = \text{FALSE}$ for any adjacent regions R_i and R_j .

where $P(R_i)$ is a logical predicate defined over the points in set R_i and $\emptyset$ is the empty set.

Region-based segmentation

- Logical predicates (also called *homogeneity criteria*) include:
 - Pure uniformity
 - Local mean relative to global mean
 - Local standard deviation relative to global mean
 - Variance
 - Texture

Region-based segmentation

- Region growing – key factors:
 - Choice of similarity criteria: for monochrome images, regions are analyzed based on intensity levels and connectivity properties.
 - Selection of seed points: these can be determined interactively (if the application allows) or based on a preliminary cluster analysis of the image, used to determine groups of pixels that share similar properties, from which a seed can be chosen.
 - Stopping rule: a region should stop growing when there are no further pixels that satisfy the homogeneity and connectivity criteria to be included in that region.

Region-based segmentation

```
Let  $f(x,y)$  be the input image
Define a set of regions  $R_1, R_2, \dots, R_n$ , each consisting of a
    single seed pixel
repeat
    for  $i = 1$  to  $n$  do
        for each pixel  $p$  at the border of  $R_i$  do
            for all neighbors of  $p$  do
                Let  $(x,y)$  be the neighbor's coordinates
                Let  $M_i$  be the mean gray level of pixels in  $R_i$ 
                if the neighbor is unassigned and
                     $|f(x,y) - M_i| \leq \Delta$  then
                    Add neighbor to  $R_i$ 
                    Update  $M_i$ 
                end if
            end for
        end for
    end for
until no more pixels can be assigned to regions
```

Region-based segmentation

- Region growing - example

$$P(R_i) = \begin{cases} \text{TRUE} & \text{if } |f(x, y) - \mu_i| \leq \Delta \\ \text{FALSE} & \text{otherwise} \end{cases}$$

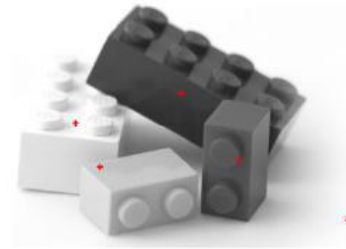
6	7	7	6	5
7	7	8	6	5
5	5	6	7	6
0	1	2	0	1
1	0	0	2	0

6	7	7	6	5
7	7	8	6	5
5	5	6	7	6
0	1	2	0	1
1	0	0	2	0

6	7	7	6	5
7	7	8	6	5
5	5	6	7	6
0	1	2	0	1
1	0	0	2	0

Region-based segmentation

- Region growing:
additional examples



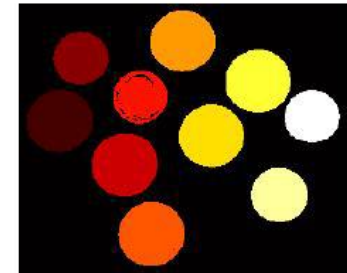
(a)



(b)



(c)



(d)

Region-based segmentation

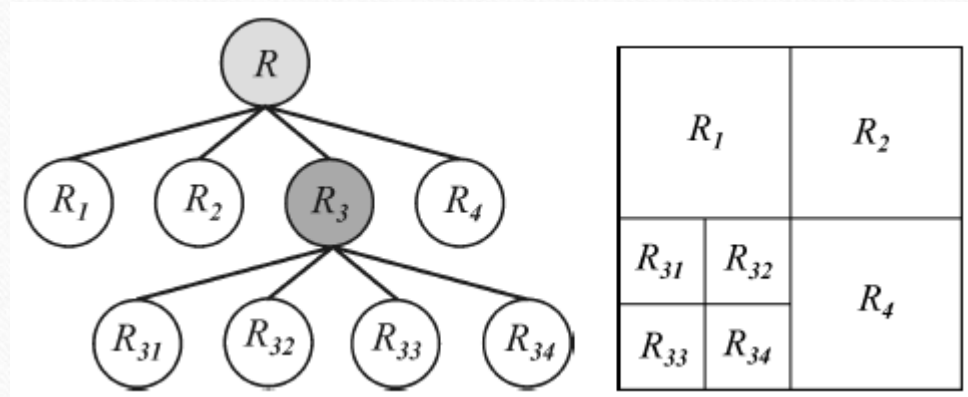
- Region growing – limitations:
 - Not very stable: significantly different results are obtained when switching between 4-connectivity and 8-connectivity criteria.
 - Segmentation results are very sensitive to choice of logical uniformity predicate.
 - The number of seeds provided by the user may not be sufficient to assign every pixel to a region.
 - If two or more seeds that should belong to the same region are incorrectly provided to the algorithm, it will be forced to create distinct regions around them whereas only one region should exist.

Region-based segmentation

- Region splitting and merging
 - Start from the entire image and partition (split) it into smaller subimages until each resulting region is considered homogeneous by some criterion.
 - Merge two or more adjacent regions into one if they satisfy a homogeneity criterion.

Region-based segmentation

- Region splitting and merging
 - Data structure: quadtree, a special type of tree in which each node (except for the leaves) has four children
 - Each leaf node in the quadtree corresponds to a region in the segmented image



Region-based segmentation

- Region splitting and merging algorithms:
 1. Define a logical uniformity predicate $P(R_i)$.
 2. Compute $P(R_i)$ for each region.
 3. Split into four disjoint quadrants any region R_i for which $P(R_i) = \text{FALSE}$.
 4. Repeat steps 2 and 3 until all resulting regions satisfy the uniformity criterion, i.e., $P(R_i) = \text{TRUE}$.
 5. Merge any adjacent regions R_j and R_k for which $P(R_j \cup R_k) = \text{TRUE}$.
 6. Repeat step 5 until no further merging is possible.

Region-based segmentation

- Morphological image segmentation algorithm (**watershed**)
- Used to represent regions in a segmented image (equivalent to catchment basins) and the boundaries among them (analogous to the ridge lines).
- In MATLAB: **watershed**

Region-based segmentation

- The distance transform
- Computes the distance from every pixel to the nearest nonzero-valued pixel.
- In MATLAB: **bwdist**

a =

0	1	1	0	1
1	1	1	0	0
0	0	0	1	0
0	0	0	0	0
0	1	0	0	0

```
>> b = bwdist(a,'cityblock')
```

b =

1	0	0	1	0
0	0	0	1	1
1	1	1	0	1
2	1	2	1	2
1	0	1	2	3

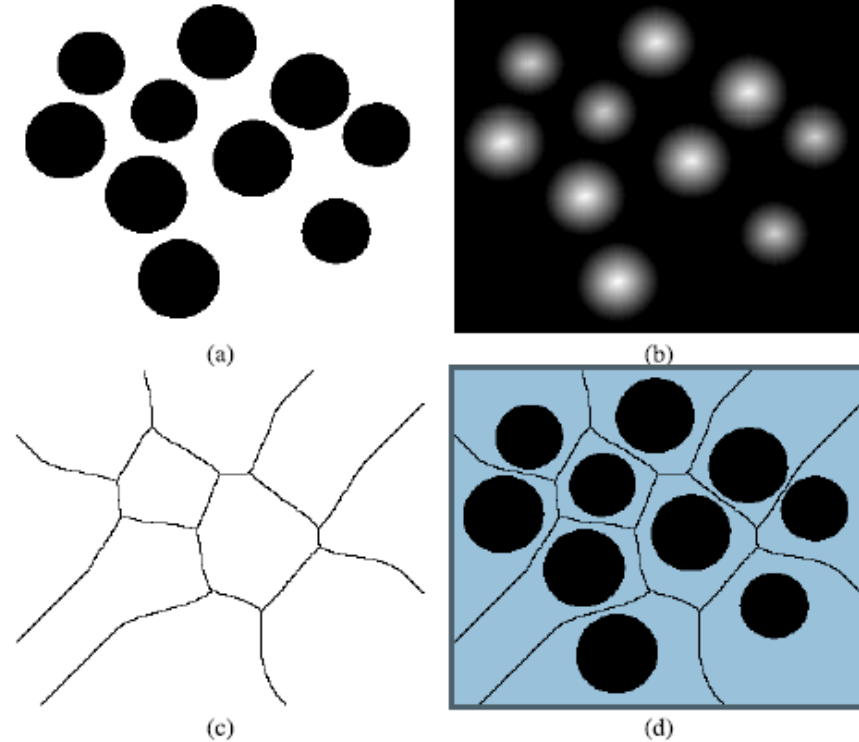
Region-based segmentation

- Morphological image segmentation algorithm (watershed) – Example 15.5

```
J = imread('Figure15_10_a.png');  
Jc = ~J;  
D = bwdist(Jc);  
L = watershed(~D);  
w = L==0;  
J2 = J & ~w;  
J3 = J | w;  
imshow(Jc), figure, imshow(~w)  
figure, imshow(mat2gray(D))  
figure, imshow(~J3)
```

Region-based segmentation

- Morphological image segmentation algorithm (watershed) – Example 15.5



Hands-on

- Tutorial 15.1: Image thresholding (page 379)